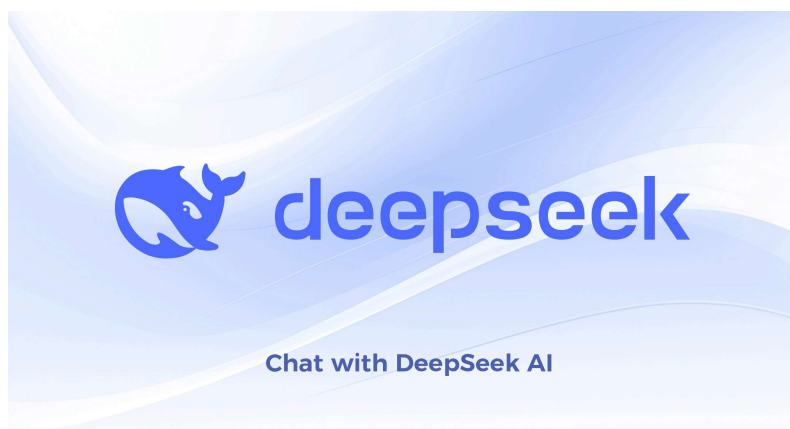


MongoDB Concepts Explained Simply - DeepSeek



Absolutely! Let's go through **MongoDB concepts one by one with practical, hands-on examples** you can actually run. I'll use the **MongoDB shell** (**mongosh**) so you can follow along.

Setup: Start MongoDB and open shell

bash

```
mongosh
use schoolDB
```

1. Database (Container for collections)

Practical:

javascript

```
show dbs
use myNewDB
db
```

2. Collection (Group of documents)

Practical:

javascript

```
db.students.insertOne({ name: "Alice", age: 22 })

db.createCollection("teachers")

show collections
```

3. Document (A single record)

Practical:

javascript

```
db.students.insertOne({
  name: "Bob",
  age: 25,
  enrolled: true,
  courses: ["Math", "Science"],
  address: { city: "NYC", zip: 10001 }
})

db.students.find().pretty()
```

4. CRUD Operations (Create, Read, Update, Delete)

CREATE

javascript

```
db.students.insertOne({ name: "Carol", age: 23, course:
"Physics" })

db.students.insertMany([
  { name: "David", age: 20, course: "Math" },
  { name: "Eva", age: 24, course: "CS" },
  { name: "Frank", age: 22, course: "Physics" }
])
```

READ

javascript

```
db.students.find()

db.students.find({ course: "Physics" })

db.students.findOne({ name: "Alice" })

db.students.find().pretty()
```

UPDATE

javascript

```
db.students.updateOne(
  { name: "Alice" },
  { $set: { age: 23 } }
)

db.students.updateMany(
  { course: "Physics" },
  { $set: { bonus: true } }
)

db.students.updateMany(
  {},
  { $inc: { age: 1 } }
)
```

DELETE

javascript

```
db.students.deleteOne({ name: "Eva" })

db.students.deleteMany({ age: { $gt: 30 } })

db.students.deleteMany({})
```

5. Query Operators (Filter with conditions)

Practical:

javascript

```
db.students.find({ age: { $gt: 22 } })

db.students.find({ age: { $lte: 21 } })

db.students.find({ course: { $in: ["Math", "CS"] } })

db.students.find({ course: "Math", age: { $gt: 20 } })

db.students.find({
  $or: [
    { course: "Math" },
    { age: { $lt: 21 } }
  ]
})

db.students.find({ course: { $ne: "Physics" } })
```

6. Projection (Choose which fields to return)

Practical:

javascript

```
db.students.find(
  { course: "Math" },
  { name: 1, age: 1, _id: 0 }
)

db.students.find({}, { age: 0 })
```

7. Index (Speed up queries)

Practical:

javascript

```
db.students.createIndex({ name: 1 })

db.students.createIndex({ course: 1, age: -1 })

db.students.getIndexes()

db.students.find({ name: "Alice" }).explain("executionStats")

db.students.dropIndex("name_1")
```

8. Aggregation Pipeline (Multi-stage data processing)

Practical:

javascript

```
db.students.aggregate([
  { $match: { age: { $gte: 22 } } },
  { $group: { _id: "$course", avgAge: { $avg: "$age" } } },
  { $sort: { avgAge: -1 } }
])

db.students.aggregate([
  { $project: { name: 1, age: 1, ageGroup: { $cond: { if: {
    $gte: ["$age", 22] }, then: "Senior", else: "Junior" } } } }
])
```

Common aggregation stages:

- **\$match** → filter
- **\$group** → group by
- **\$sort** → sort

- **\$project** → reshape documents
- **\$limit** / **\$skip** → pagination
- **\$unwind** → flatten arrays

9. Embedding vs Referencing (Real example)

Embedding (store inside)

javascript

```
db.users.insertOne({
  name: "Alice",
  email: "alice@example.com",
  orders: [
    { product: "Laptop", price: 1200 },
    { product: "Mouse", price: 25 }
  ]
})

db.users.findOne({ name: "Alice" })
```

Referencing (store IDs)

javascript

```
db.users.insertOne({
  _id: 101,
  name: "Bob",
  email: "bob@example.com"
})

db.orders.insertMany([
  { userId: 101, product: "Phone", price: 800 },
  { userId: 101, product: "Case", price: 20 }
])

db.users.aggregate([
  {
    $lookup: {
      from: "orders",
      localField: "_id",
      foreignField: "userId",
      as: "userOrders"
    }
  }
])
```

10. Sorting, Limiting, Skipping (Pagination)

Practical:

javascript

```
db.students.find().sort({ age: 1 })

db.students.find().limit(2)

db.students.find().skip(2).limit(2)

db.students.find().sort({ age: -1 }).limit(3)
```

11. Count Documents

Practical:

javascript

```
db.students.countDocuments()  
db.students.countDocuments({ course: "Math" })
```

12. Bulk Write (Multiple operations at once)

Practical:

javascript

```
db.students.bulkWrite([  
  { insertOne: { document: { name: "Grace", age: 21 } } },  
  { updateOne: { filter: { name: "Alice" }, update: { $set: {  
    age: 24 } } } },  
  { deleteOne: { filter: { name: "Frank" } } }  
)
```

13. Transactions (All-or-nothing operations)

Practical:

javascript

```
const session = db.getMongo().startSession()  
session.startTransaction()  
  
try {  
  const students = session.getDatabase("schoolDB").students  
  
  students.updateOne({ name: "Alice" }, { $inc: { balance:  
-100 } })  
  students.updateOne({ name: "Bob" }, { $inc: { balance: 100 }  
})  
  
  session.commitTransaction()  
  print("Transaction successful")  
} catch(e) {  
  session.abortTransaction()  
  print("Transaction rolled back")  
} finally {  
  session.endSession()  
}
```

14. Change Streams (Real-time notifications)

Practical:

javascript

```
const changeStream = db.students.watch()
changeStream.on("change", (change) => {
  print("Change detected:", change.operationType)
})

db.students.insertOne({ name: "New Student", age: 30 })
```

15. Useful Practical Commands

javascript

```
db.students.drop()

db.dropDatabase()

db.students.stats()
```

Complete Practical Workflow Example

javascript

```
use ecommerce
db.products.insertMany([
  { name: "Laptop", price: 1200, category: "Electronics",
    stock: 5 },
  { name: "Mouse", price: 25, category: "Electronics", stock:
    100 },
  { name: "Book", price: 15, category: "Books", stock: 200 }
])

db.products.find({ price: { $lt: 100 } })

db.products.updateOne({ name: "Laptop" }, { $inc: { stock: -1
  } })

db.products.createIndex({ category: 1 })

db.products.aggregate([
  { $group: { _id: "$category", avgPrice: { $avg: "$price" } }
  }
])

db.products.deleteOne({ stock: 0 })
```

Quick Cheat Sheet for Daily Use

Want me to show you a real mini-project (like a blog database or task manager) using all these concepts step by step? Just say the word!

all command from basic to advanced

MongoDB Complete Command Guide: Basic to Advanced

I'll organize commands from **beginner** → **intermediate** → **advanced** with practical examples you can run.

PART 1: BASIC COMMANDS (Daily Operations)

Database Commands

javascript

```
show dbs
show databases

use myDB

db

db.stats()

db.dropDatabase()
```

Collection Commands

javascript

```
show collections
show tables

db.createCollection("users")
db.createCollection("logs", { capped: true, size: 100000, max:
5000 })

db.users.stats()
db.users.totalSize()

db.users.renameCollection("customers")

db.users.drop()
```

CRUD Basic Operations

CREATE

javascript

```
db.users.insertOne({ name: "John", age: 25 })

db.users.insertMany([
  { name: "Alice", age: 30 },
  { name: "Bob", age: 28 }
])

db.users.insertOne({ _id: 100, name: "Custom", age: 35 })

db.users.insert({ name: "Old", age: 40 })
```

READ

javascript

```
db.users.find()

db.users.findOne({ name: "John" })

db.users.find().pretty()

db.users.find({ age: 25 })

db.users.find({}, { name: 1, _id: 0 })
db.users.find({ age: 25 }, { name: 1, age: 1 })

db.users.countDocuments()
db.users.countDocuments({ age: { $gt: 25 } })
db.users.estimatedDocumentCount()

db.users.distinct("age")
db.users.distinct("name", { age: { $gt: 25 } })
```

UPDATE

javascript

```
db.users.updateOne(
  { name: "John" },
  { $set: { age: 26 } }
)

db.users.updateMany(
  { age: { $lt: 30 } },
  { $inc: { age: 1 } }
)

db.users.replaceOne(
  { name: "John" },
  { name: "John", age: 30, city: "NYC" }
)

db.users.updateOne(
  { name: "Mary" },
  { $set: { age: 25 } },
  { upsert: true }
)

db.users.findAndModify({
  query: { name: "Bob" },
  update: { $inc: { age: 1 } },
  new: true
})
```

DELETE

javascript

```
db.users.deleteOne({ name: "John" })

db.users.deleteMany({ age: { $lt: 18 } })

db.users.deleteMany({})

db.users.remove({ age: 30 })
```

PART 2: QUERY OPERATORS

Comparison Operators

javascript

```
$eq
$ne
$gt
$gte
$lt
$lte
$in
$nin

db.users.find({ age: { $eq: 25 } })
db.users.find({ age: { $gt: 20, $lt: 30 } })
db.users.find({ name: { $in: ["John", "Alice"] } })
db.users.find({ age: { $nin: [20, 25, 30] } })
```

Logical Operators

javascript

```
$and
$or
$not
$nor

db.users.find({ $and: [{ age: { $gt: 20 } }, { city: "NYC" }] })
db.users.find({ $or: [{ age: 25 }, { name: "John" }] })
db.users.find({ age: { $not: { $gt: 25 } } })
db.users.find({ $nor: [{ age: 25 }, { city: "LA" }] })
```

Element Operators

javascript

```
$exists
$type

db.users.find({ email: { $exists: true } })
db.users.find({ age: { $type: "number" } })
db.users.find({ age: { $type: 16 } })
```

Array Operators

javascript

```
$all
$size
$elemMatch

db.users.find({ hobbies: { $all: ["reading", "sports"] } })
db.users.find({ hobbies: { $size: 3 } })
db.users.find({
  results: {
    $elemMatch: { score: { $gt: 80 }, subject: "Math" }
  }
})
```

Evaluation Operators

javascript

```
$regex
$expr
$mod
$where

db.users.find({ name: { $regex: /^J/, $options: "i" } })
db.users.find({ $expr: { $gt: ["$age", "$minAge"] } })
db.users.find({ age: { $mod: [2, 0] } })
```

PART 3: AGGREGATION PIPELINE

Basic Stages

javascript

```
db.orders.aggregate([
  { $match: { status: "completed" } }
])

db.orders.aggregate([
  { $group: { _id: "$product", total: { $sum: 1 } } }
])

db.users.aggregate([
  { $project: { fullName: "$name", age: 1, _id: 0 } }
])

db.users.aggregate([
  { $sort: { age: -1, name: 1 } }
])

db.users.aggregate([
  { $sort: { age: -1 } },
  { $skip: 10 },
  { $limit: 5 }
])
```

Array Stages

javascript

```
db.users.aggregate([
  { $unwind: "$hobbies" },
  { $group: { _id: "$hobbies", count: { $sum: 1 } } }
])

db.users.aggregate([
  { $unwind: { path: "$hobbies", preserveNullAndEmptyArrays:
true } }
])
```

Lookup (Join)

javascript

```
db.orders.aggregate([
  {
    $lookup: {
      from: "users",
      localField: "userId",
      foreignField: "_id",
      as: "user"
    }
  }
])

db.users.aggregate([
  {
    $lookup: {
      from: "orders",
      let: { userId: "$_id" },
      pipeline: [
        { $match: { $expr: { $eq: ["$$userId", "$userId"] } } },
        { $limit: 5 }
      ],
      as: "recentOrders"
    }
  }
])
```

Group Accumulators

javascript

```
$sum
$avg
$min
$max
$first
$last
$push
$addToSet

db.sales.aggregate([
  {
    $group: {
      _id: "$product",
      totalRevenue: { $sum: "$amount" },
      avgPrice: { $avg: "$price" },
      minPrice: { $min: "$price" },
      maxPrice: { $max: "$price" },
      allPrices: { $push: "$price" },
      uniquePrices: { $addToSet: "$price" }
    }
  }
])
```

Advanced Stages

javascript

```
db.orders.aggregate([
  {
    $facet: {
      totalSales: [{ $group: { _id: null, total: { $sum:
"$amount" } } } ]},
      byProduct: [{ $group: { _id: "$product", total: { $sum:
"$amount" } } } ]},
      byMonth: [
        { $group: { _id: { $month: "$date" }, total: { $sum:
"$amount" } } }
      ]
    }
  }
])
```

```
db.products.aggregate([
  {
    $bucket: {
      groupBy: "$price",
      boundaries: [0, 100, 500, 1000],
      default: "Other",
      output: { count: { $sum: 1 } }
    }
  }
])
```

```
db.products.aggregate([
  { $bucketAuto: { groupBy: "$price", buckets: 5 } }
])
```

```
db.products.aggregate([
  { $sortByCount: "$category" }
])
```

```
db.users.aggregate([
  { $addFields: { ageGroup: { $cond: { if: { $gte: ["$age",
18] }, then: "Adult", else: "Minor" } } } }
])
```

```
db.users.aggregate([
  { $replaceRoot: { newRoot: "$address" } }
])
```

```
db.orders.aggregate([
  { $group: { _id: "$product", total: { $sum: "$amount" } } },
  { $merge: { into: "productTotals", whenMatched: "replace" } }
])
```

Date Operators

javascript

```
$year, $month, $dayOfMonth, $hour, $minute, $second
$dateToString
$dateAdd, $dateSubtract

db.orders.aggregate([
  {
    $project: {
      year: { $year: "$orderDate" },
      month: { $month: "$orderDate" },
      formatted: { $dateToString: { format: "%Y-%m-%d", date:
"$orderDate" } }
    }
  }
])
```

String Operators

javascript

```
$concat, $substr, $toUpper, $toLower, $trim
$split, $strLenBytes

db.users.aggregate([
  {
    $project: {
      fullName: { $concat: ["$firstName", " ", "$lastName"] },
      firstChar: { $substr: ["$name", 0, 1] }
    }
  }
])
```

PART 4: INDEXING

Index Types

javascript

```
db.users.createIndex({ name: 1 })
db.users.createIndex({ age: -1 })

db.users.createIndex({ name: 1, age: -1 })

db.users.createIndex({ email: 1 }, { unique: true })

db.articles.createIndex({ title: "text", content: "text" })

db.users.createIndex({ userId: "hashed" })

db.places.createIndex({ location: "2dsphere" })

db.users.createIndex(
  { email: 1 },
  { partialFilterExpression: { email: { $exists: true } } }
)

db.sessions.createIndex(
  { createdAt: 1 },
  { expireAfterSeconds: 3600 }
)

db.users.createIndex({ email: 1 }, { sparse: true })
```

Index Management

javascript

```
db.users.getIndexes()

db.users.dropIndex("name_1")
db.users.dropIndexes()

db.users.hideIndex("name_1")
db.users.unhideIndex("name_1")
```

Query Optimization

javascript

```
db.users.find({ name: "John" }).explain()
db.users.find({ age: { $gt: 25 } }).explain("executionStats")
db.users.find({ name: "John" }).explain("allPlansExecution")

db.users.find({ name: "John", age: 25 }).hint({ name: 1, age:
-1 })
```

PART 5: ADVANCED OPERATIONS

Bulk Operations

javascript

```
var bulk = db.users.initializeUnorderedBulkOp()

bulk.insert({ name: "User1", age: 20 })
bulk.find({ name: "User2" }).update({ $set: { age: 25 } })
bulk.find({ name: "User3" }).remove()
bulk.execute()

var bulk = db.users.initializeOrderedBulkOp()
```

Transactions

javascript

```
const session = db.getMongo().startSession()
session.startTransaction()

try {
  const users = session.getDatabase("myDB").users
  const accounts = session.getDatabase("myDB").accounts

  users.updateOne({ name: "Alice" }, { $inc: { balance: -100 } })
  accounts.updateOne({ name: "Bob" }, { $inc: { balance: 100 } })

  session.commitTransaction()
  print("Transaction committed")
} catch(e) {
  session.abortTransaction()
  print("Transaction aborted: " + e)
} finally {
  session.endSession()
}
```

Change Streams

javascript

```
const changeStream = db.users.watch()
changeStream.on("change", (change) => {
  print(JSON.stringify(change))
})

const pipeline = [{ $match: { "operationType": "insert" } }]
const changeStream = db.users.watch(pipeline)

const changeStream = db.watch()

changeStream.close()
```

MapReduce

javascript

```
db.orders.mapReduce(  
  function() { emit(this.customerId, this.amount); },  
  function(key, values) { return Array.sum(values); },  
  {  
    query: { status: "completed" },  
    out: "customerTotals"  
  }  
)
```

PART 6: ADMINISTRATION COMMANDS

User Management

javascript

```
db.createUser({  
  user: "admin",  
  pwd: "password123",  
  roles: ["readWrite", "dbAdmin"]  
})  
  
db.createUser({  
  user: "reporter",  
  pwd: "report123",  
  roles: [{ role: "read", db: "myDB" }]  
})  
  
db.getUsers()  
db.getUser("admin")  
  
db.updateUser("admin", { roles: ["readWrite"] })  
  
db.changeUserPassword("admin", "newPass123")  
  
db.dropUser("reporter")
```

Role Management

javascript

```
db.createRole({
  role: "manageOrders",
  privileges: [
    { resource: { db: "myDB", collection: "orders" }, actions:
["find", "update"] }
  ],
  roles: []
})

db.grantRolesToUser("admin", ["manageOrders"])

db.revokeRolesFromUser("admin", ["manageOrders"])
```

Server Status

javascript

```
db.serverStatus()
db.serverStatus().connections
db.serverStatus().opcounters

db.stats()
db.printCollectionStats()

rs.status()
rs.conf()

sh.status()
db.printShardingStatus()
```

Backup & Restore

javascript

```
mongodump --db myDB --out /backup/  
mongorestore --db myDB /backup/myDB/  
  
mongoexport --db myDB --collection users --out users.json  
mongoimport --db myDB --collection users --file users.json  
  
mongoexport --db myDB --collection users --query '{"age":  
{ "$gt": 18 } }' --out adults.json
```

Profiling

javascript

```
db.setProfilingLevel(1, { slowms: 100 })  
  
db.getProfilingLevel()  
  
db.system.profile.find().pretty()  
db.system.profile.find({ op: "query" }).sort({ ts: -1  
}).limit(5)  
  
db.setProfilingLevel(0)
```

PART 7: UTILITY COMMANDS

Data Manipulation

javascript

```
db.users.aggregate([ { $out: "users_backup" } ] )

db.users.find().forEach(d => db.users_clone.insert(d))

db.users.updateMany(
  {},
  { $set: { address: { city: "$city", zip: "$zipCode" } } }
)
```

Validation

javascript

```
db.createCollection("users", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "age"],
      properties: {
        name: { bsonType: "string" },
        age: { bsonType: "int", minimum: 0, maximum: 120 }
      }
    }
  }
})

db.runCommand({
  collMod: "users",
  validator: { age: { $type: "number" } }
})
```

Checkpoints & Logs

javascript

```
db.fsyncLock()  
db.fsyncUnlock()  
  
db.adminCommand({ getLog: "global" })  
db.adminCommand({ getLog: "startupWarnings" })  
  
db.adminCommand({ logRotate: 1 })
```

QUICK REFERENCE CARD

Most Used Commands

javascript

```
use dbName  
show dbs  
db.dropDatabase()  
  
show collections  
db.createCollection("name")  
db.collection.drop()  
  
db.collection.insertOne({})  
db.collection.find({})  
db.collection.updateOne({}, {$set: {}})  
db.collection.deleteOne({})  
  
db.collection.aggregate([{$match: {}}])  
  
db.collection.createIndex({field: 1})  
  
db.collection.stats()  
db.collection.countDocuments()
```

Essential Operators

javascript

```
$gt, $lt, $gte, $lte, $eq, $ne, $in, $nin  
$and, $or, $not, $nor  
$exists, $type  
$regex
```

```
$set, $unset, $inc, $mul  
$push, $pull, $addToSet  
$rename, $min, $max
```

```
$match, $group, $project, $sort, $limit  
$unwind, $lookup, $addFields  
$sum, $avg, $min, $max, $push
```

Want me to create a specific cheat sheet for your use case (e.g., e-commerce, analytics, real-time app)? Or show you how to optimize a specific query?